EP1 MAC0422

Analisando o Escalonador de Processos

Kaiky Henrique Ribeiro Cintra - 13731160

ROTEIRO

1. Determinismo
2. Experimentos
3. Referências

Determinismo

O escalonador de processos não é determinístico! Minha implementação executa as threads em uma ordem definida mas sua ordem de encerramento e posterior reordenação não pode ser garantida por diversos fatores, como disputa de CPU, variações de tempo na memória, etc.

Experimentos (1 thread)

p1 5 0 1
p2 5 1 2
p3 5 3 1
p4 10 4 3
p5 10 5 1
p6 20 6 3
p7 25 7 2
p8 25 7 2
p9 30 10 1
p10 30 10 3

A entrada tem tarefas com tempos de execução próximos que se amontoam em um curto espaço de tempo.

Inesperado!

SJF>RR

Nesse caso processos “demorados” podem ser prejudicados por Round Robin, comprometendo as deadlines. Por outro lado, o SJF pode manter a expectativa de execução razoável e cumprir os deadlines.

SJF:

1 p1 1.0 1.0

1 p2 3.1 2.1

1 p3 4.3 1.3

1 p4 7.4 3.4

1 p5 8.6 3.6

1 p6 16.9 10.9

1 p7 10.6 3.6

1 p8 13.8 6.8

1 p9 11.7 1.7

1 p10 20.0 10.0

ROUND ROBIN:

1 p1 1.7 1.7

1 p2 3.3 2.3

1 p3 5.0 2.0

0 p4 17.3 13.3

0 p5 10.3 5.3

0 p6 20.5 14.5

1 p7 18.5 11.5

1 p8 19.1 12.1

1 p9 19.3 9.3

1 p10 22.3 12.3

32

A entrada tem tarefas com tempos de execução muito variados e tempos de início espaçados. É esperado que o SJF cause starvation sem preempção.

p1 10 0 15
p2 10 10 1
p3 20 14 1
p4 30 18 15
p5 35 22 1
p6 35 23 1
p7 35 24 1
p8 35 25 1
p9 35 26 1
p10 35 27 1

COMO ESPERADO RR>SJF

O ROUND ROBIN equilibra melhor o conjunto de processos em que alguns são muito mais demorados que outros, impedindo que a execução seja travada

SJF:

0 p1 15.0 15.0
0 p2 16.2 6.2
1 p3 17.4 3.4
0 p4 33.0 15.0
1 p5 34.2 12.2
0 p6 35.4 12.4
0 p7 36.5 12.5
0 p8 37.7 12.7
0 p9 38.8 12.8
0 p10 39.9 12.9

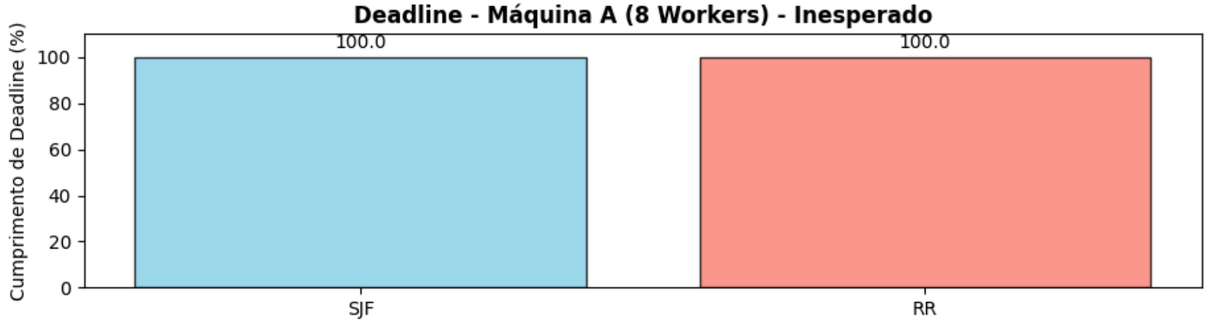
ROUND ROBIN:

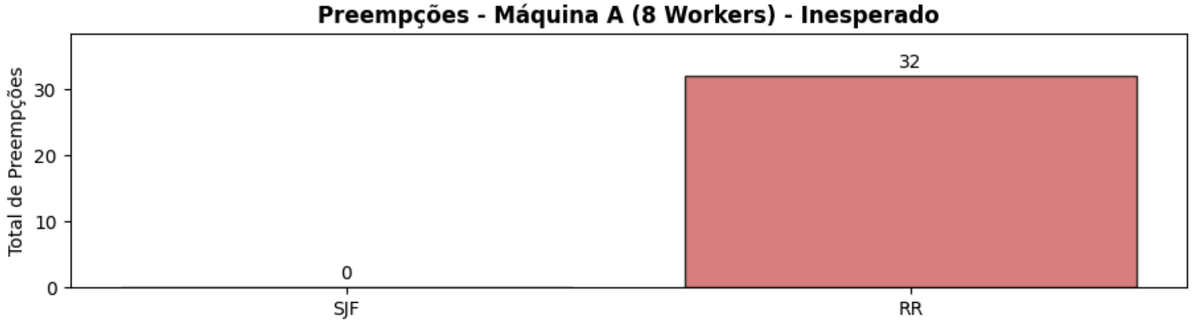
0 p1 20.2 20.2
0 p2 12.8 2.8
1 p3 16.7 2.7
0 p4 43.4 25.4
1 p5 25.3 3.3
1 p6 28.9 5.9
1 p7 29.1 5.1
1 p8 31.5 6.5
1 p9 31.7 5.7
1 p10 31.9 4.9

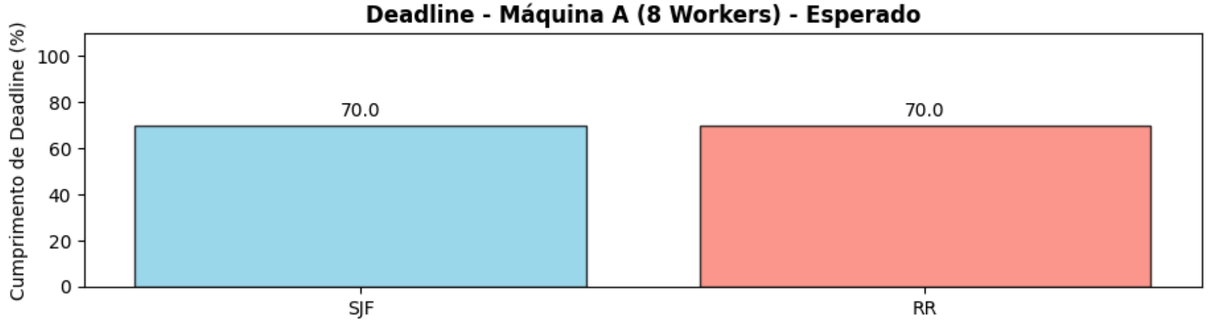
Gráficos

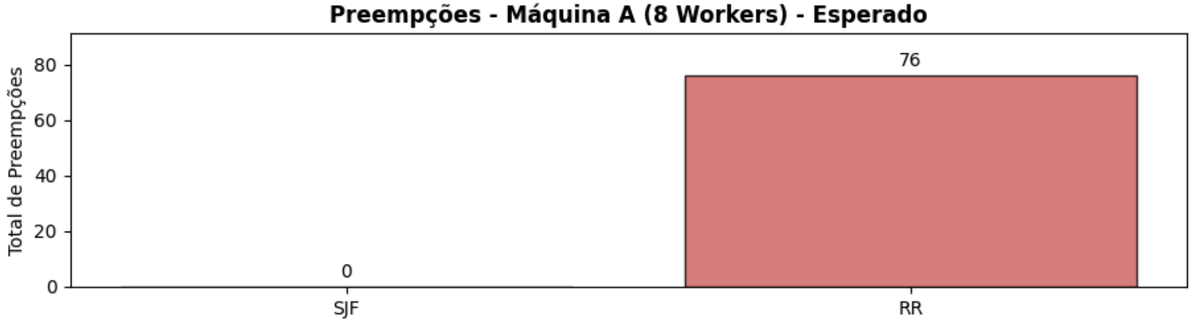
Em vez de testar máquinas diferentes eu programei o escalonador para trabalhar com N worker threads que podem ser distribuídos em vários núcleos de processamento pelo SO. A ideia é que estamos interessados em testar o caso em que temos um 1 < número de threads < número de núcleos, dessa forma podemos ver parte do ganho de implementar paralelismo.

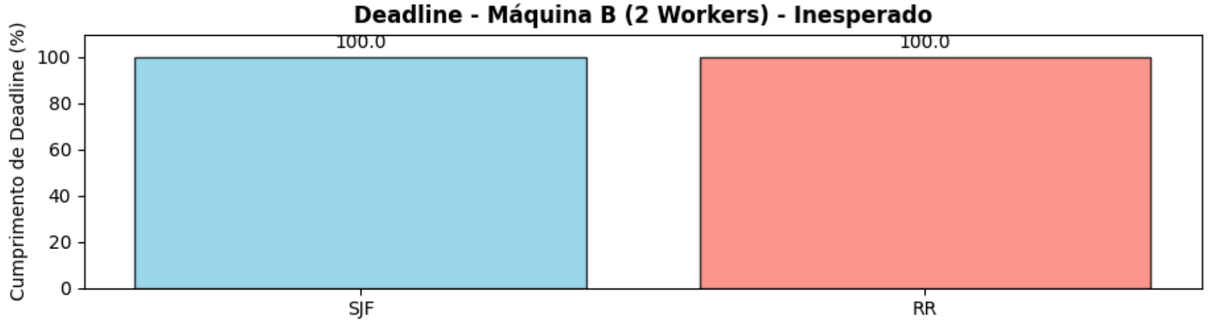
Os dados para os gráficos foram gerados rodando o programa com <u>2 workers e 8 workers em uma máquina com 16 núcleos</u>.

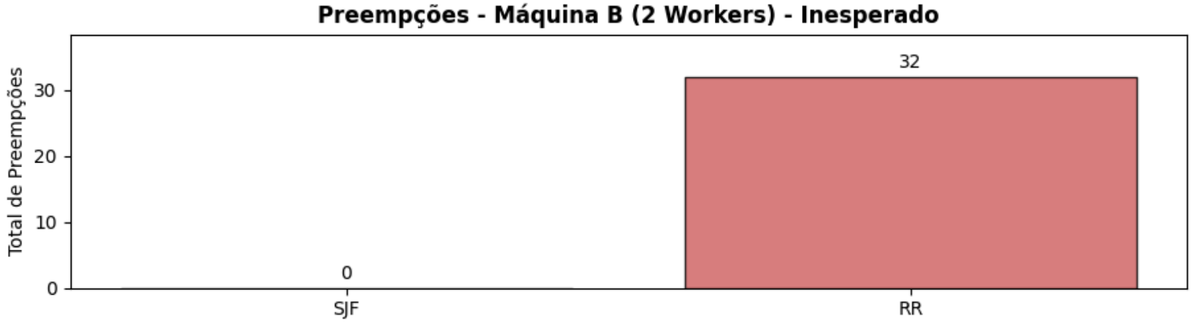

Deadline - Máquina A (8 Workers) - Inesperado
Cumprimento de Deadline (%)
100.0
100.0
SJF
RR

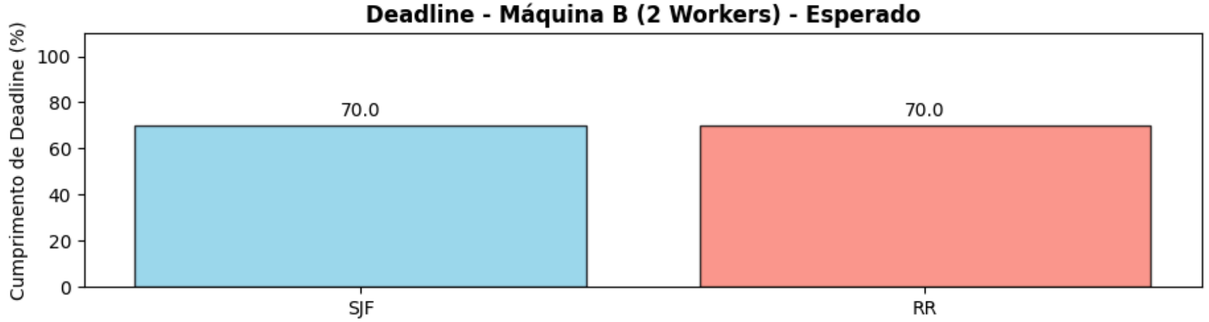

Preempções - Máquina A (8 Workers) - Inesperado
Total de Preempções
0
32
SJF
RR

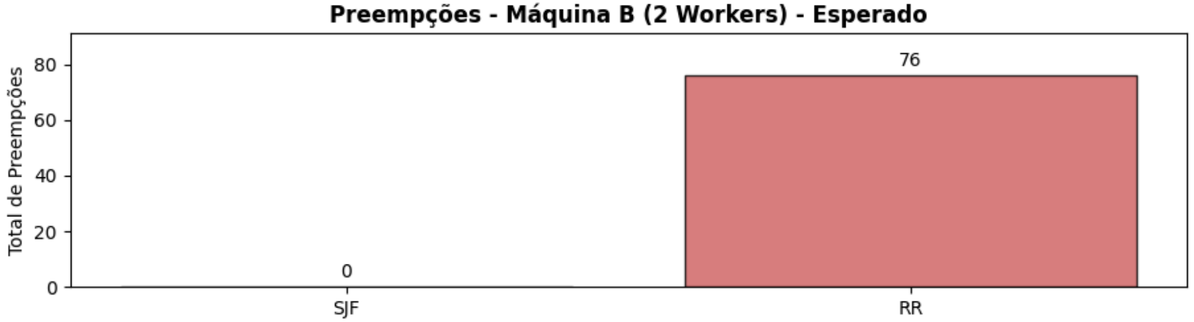

Deadline - Máquina A (8 Workers) - Esperado
Cumprimento de Deadline (%)
70.0
70.0
SJF
RR


Preempções - Máquina A (8 Workers) - Esperado
Total de Preempções
0
76
SJF
RR


Deadline - Máquina B (2 Workers) - Inesperado
Cumprimento de Deadline (%)
100.0
100.0
SJF
RR


Preempções - Máquina B (2 Workers) - Inesperado
Total de Preempções
0
32
SJF
RR


Deadline - Máquina B (2 Workers) - Esperado
Cumprimento de Deadline (%)
70.0
70.0
SJF
RR


Preempções - Máquina B (2 Workers) - Esperado
Total de Preempções
0
76
SJF
RR

Infelizmente sem o escalonamento por prioridade os dados não tiveram muita variação, o que também foi causado pelo conjunto de entrada de resultado esperado e inesperado.

Os tempos de cada processo tiveram bastante variação em cada versão do escalonador usada para gerar os dados. Esses dados completos podem ser vistos no repositório, junto com o código Python usado para gerar os gráficos.

Referências

Eu anotei todas as referências usadas em ordem, além de adicioná-las em commits ao meu repositório do EP:

https://utat-ss.readthedocs.io/en/master/c-programming/c_h_files.html
https://utat-ss.readthedocs.io/en/master/c-programming/header-files.html#define
https://makefiletutorial.com/
https://www.w3schools.com/c/c_strings.php
man date
https://unix.stackexchange.com/questions/11454/what-is-the-difference-between-a-builtin-command-and-one-that-is-not
https://www.w3schools.com/c/c_booleans.php
https://www.geeksforgeeks.org/c/sleep-function-in-c/
https://stackoverflow.com/questions/7965437/undefined-reference-to-main-collect2-ld-returned-1-exit-status
https://web.mit.edu/gnu/doc/html/rlman_2.html
https://stackoverflow.com/questions/3783149/how-do-i-properly-reference-the-gnu-readline-library-to-scan-terminal-input
https://stackoverflow.com/questions/38792542/readline-h-history-usage-in-c
https://tiswww.case.edu/php/chet/readline/readline.html
https://en.wikibooks.org/wiki/C_Programming/String_manipulation
man syscalls
https://www.ime.usp.br/~kon/MAC211/syscalls.html
man getuid, whoami, id
https://www.geeksforgeeks.org/c/exec-family-of-functions-in-c/
https://www.geeksforgeeks.org/c/wait-system-call-c/
https://man7.org/linux/man-pages/man2/gethostname.2.html
uilding-a-multi-threaded-task-queue-with-pthreads/
man gethostname, uname,
https://www.w3schools.com/c/c_structs.php
https://stackoverflow.com/questions/26717709/how-we-can-call-a-system-function-like-pwd-or-ls-l-without-using-system-or-ex
man getcwd

https://stackoverflow.com/questions/9449241/where-is-path-max-defined-in-linux
https://stackoverflow.com/questions/61899571/my-syscall-cause-segmentation-fault-core-dumped
man calloc
https://man7.org/linux/man-pages/man2/execve.2.html
https://www.geeksforgeeks.org/c/array-of-strings-in-c/
https://www.w3schools.com/c/c_switch.php
https://stackoverflow.com/questions/32942123/substring-from-index-1-until-the-end-of-string
https://stackoverflow.com/questions/4513316/split-string-in-c-every-white-space
https://stackoverflow.com/questions/4160273/strtok-with-space-delimiter
https://filippo.io/linux-syscall-table/
man signal, kill(2)
https://www.ic.unicamp.br/~celio/mc514/linux/linux_pgsignals.html
https://www.cyberciti.biz/faq/bash-get-exit-code-of-command/
https://serverfault.com/questions/366474/whats-a-proper-way-of-checking-if-a-pid-is-running
https://www.geeksforgeeks.org/c/convert-string-to-int-in-c/
https://man7.org/linux/man-pages/man3/errno.3.html
https://www.geeksforgeeks.org/c/thread-functions-in-c-c/
https://sites.cs.ucsb.edu/~rich/class/old.cs170/notes/Threads/index.html
https://stackoverflow.com/questions/72625285/preempt-a-hot-loop-thread-from-user-space
https://pubs.opengroup.org/onlinepubs/009604599/functions/pthread_cond_timedwait.html
https://stackoverflow.com/questions/3501338/c-read-file-line-by-line
https://techfortalk.co.uk/2025/03/08/b

O Gemini (IA) foi usado para depurar alguns detalhes do escalonador e gerar o gráfico do relatório. A conversa está aqui

Link para o repositório, onde o processo de escrita de código pode ser verificado.